MUTAGENT

Agentic Development Lifecycle · Hackathon Quickstart

Build an AI agent, prove it works, find out why it fails, and fix it — all from Helix — one conversational orchestrator that routes to specialized subagents. This guide takes you from clone to your first agent.

① Spec ② Build ③ Evaluate ④ Diagnose ⑤ Improve

🏆 The Challenge — build the most sophisticated agent, prove it works

Build & evaluate the most sophisticated AI agent you can

End-to-end through the Mutagent framework, in any harness/framework — **Mastra · LangGraph · Claude Code · Codex**. Run `*spec → *build → *evaluate` and prove it works with a real eval set. The more ambitious and capable — real jobs, tools, integrations, triggers — the better.

🏆 **BONUS · Extend the Lifecycle** add a new ADL stage / `*command` / skill that cleanly fits Helix

Judging Criteria

1. **Sophistication of the agent** — ambition & complexity: jobs, tools, triggers, real integrations
2. **Loop completeness** — `*spec → *build → *evaluate` (diagnose/improve count more)
3. **Proof it works** — eval criteria + a real dataset (≥20) + a scorecard
- 🏆 **Framework extension** *(bonus)* — does your new command/stage work + fit?

Delivery

Agent code left on the `mutagent-hackathon` codebase.

Orchestrator (Helix) + subagent session transcripts — required.

These *are* your submission — they serve as both **framework feedback** and **proof you used the system**. 😉

① Quick Start — zero to orchestrator in 3 steps

1 Clone

Grab the public hackathon repo.

```
$ git clone github.com/mutagent-io/
  mutagent-hackathon
$ cd mutagent-hackathon
```

2 Install the system

Installs the agents + skills into `.claude/` and `.codex/`.

```
$ bunx @mutagent/helix init
  # or npx / pnpx
```

3 Boot

Start your coding agent, then summon the orchestrator.

```
$ claude   # or codex
❯ mutagent
→ dashboard · lifecycle ·
  what you can do
```

What you'll see — the booted orchestrator

```
❯ mutagent

🧬  M U T A G E N T   · ADL Orchestrator — Helix routes to your subagents
──────────────────────────────────────────────────────────────────────
  LIFECYCLE   ① SPEC → ② BUILD → ③ EVALUATE → ④ DIAGNOSE → ⑤ IMPROVE
  SYSTEM      agentspec · skill-builder · evaluator · diagnostics
  SETUP       ⚠ not onboarded yet — run *onboard
──────────────────────────────────────────────────────────────────────
  COMMANDS    *spec   *build   *evaluate   *diagnose   *onboard   *status
  Type a *command — or just say what you want.
```

How it works — one orchestrator, many subagents

Helix is the one orchestrator — it routes each stage to a specialized subagent. Jump in anywhere; nothing moves on without you.

① SPEC → ② BUILD → ③ EVALUATE → ④ DIAGNOSE → ⑤ IMPROVE … then loops back to ② build — until it passes

Drive the lifecycle — say a stage, or just what you want

① SPEC	`*spec`	describe your agent → a validated spec file
② BUILD	`*build`	turn the spec into a working agent
③ EVALUATE	`*evaluate`	check how good it is — pass / fail per behavior
④ DIAGNOSE	`*diagnose`	find out why it failed — ranked fixes
⑤ IMPROVE	`gated`	apply the fix, then re-check — until it passes

First time? — `*onboard`

Add your provider keys, pick your workspace, and choose your models. Then jump into any stage.

> 💬 YOU CAN SAY
>
> “set me up with my Anthropic and GitHub keys”
>
> “what agents and skills do we have?”

Two ways to drive it: type a `*command`, or just say what you want — Helix routes it to the right stage and waits for you.

③ Spec — turn an idea into a working spec

A short guided interview captures **what** your agent is, then writes a spec file you can build from. You answer with quick pick-lists.

```
❯ *spec
🧬 What agent do you want to build?
❯ A Mastra agent that triages our support
  inbox and drafts replies for approval.
🧬 ⟨pick-lists⟩ what kind? · how should it
  talk to your tools?
❯ ⟨taps options⟩
🧬 wrote your spec ✓ — next: *build
  (I won't move on without you)
```

The flow

Describe it → answer a few forks → get a validated spec. Edit the spec later and re-build; changes flow one way (spec → agent).

💬 **YOU CAN SAY**

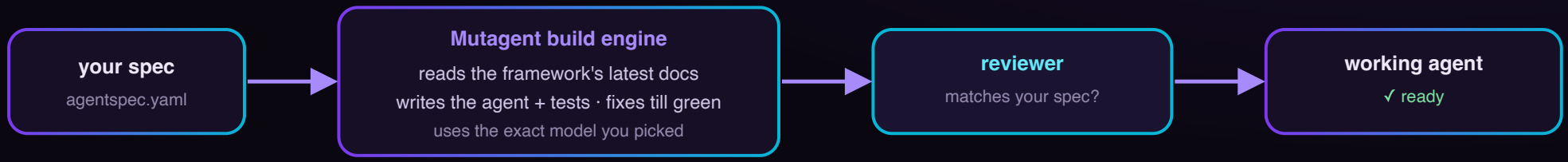
“design a new agent that triages our support inbox”

“spec out a research agent that benchmarks our models”

“I want to build an agent that reviews pull requests”

📦 PRODUCES **your agent spec · agentspec.yaml**

④ Build — your spec becomes a real, working agent



What the reviewer checks

A second pass confirms the build is faithful to your spec — every capability you asked for is actually there — before you move on. No silent gaps.

💬 YOU CAN SAY

“build the agent I just spec'd”

“implement my support-triage spec into Mastra”

“generate the code for the support-triage agent”

📦 PRODUCES **a working agent (the code) + a build report**

⑤ Evaluate — four ways to know if your agent is good

FIND Find new evals

Looks at your agent's runs and suggests what's worth testing.

> "find evals for the PR-review agent from its recent runs"

📦 **suggested evals + a starter test set**

SCORE Evaluate it

Scores your agent's runs → a clear **pass / fail** per behavior.
Built-in judge (no setup) · or a test-suite that runs in your own CI.

> "how good is my support agent on its last 50 conversations?"

📦 **a scorecard (+ a test-suite, your-stack)**

CREATE Create evals

Build the checks yourself, guided, based on your spec.

> "add an eval that checks it never sends a duplicate reply"

📦 **your eval suite (the checks)**

DATA Make a test set

Turn a few examples into a realistic set of test cases.

> "generate tricky test cases for the PR-review agent"

📦 **a dataset of test cases**

⑥ Diagnose — when an eval fails, find out why

It explains — it doesn't change anything yet

Takes the failures from evaluate and works out the root cause: what broke, where in the code, and a ranked list of fixes (★ = the one it recommends).

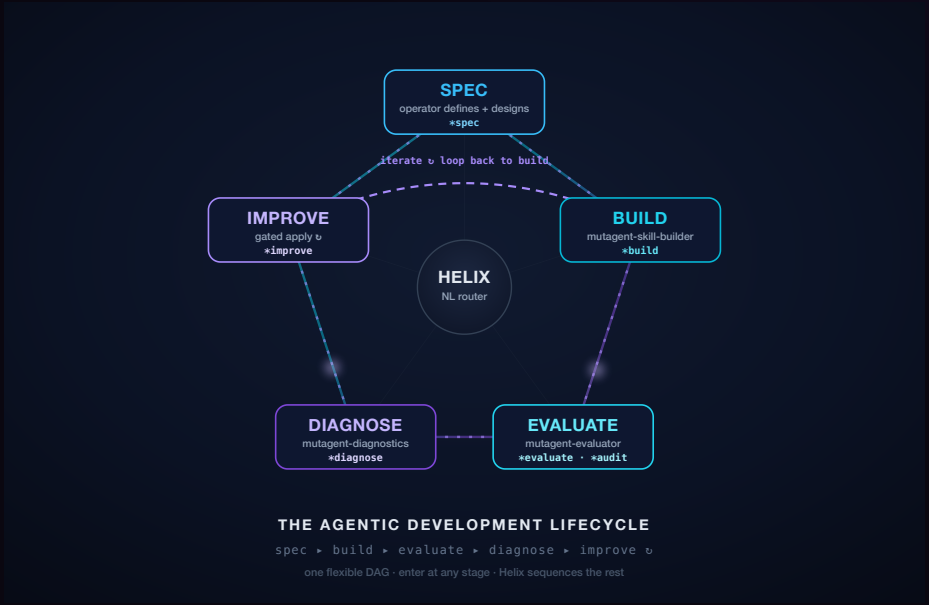
💬 YOU CAN SAY

“why did the support agent fail its escalation eval?”
“what's wrong with my agent's approval flow?”
“diagnose the failures from the last evaluation run”

📦 PRODUCES **a diagnosis report · what · why · ranked fixes**

⑦ Improve — apply the fix, then re-check

The **EDD loop** is the lifecycle's tight inner cycle — **build → evaluate → diagnose → improve → back to build** — running until the eval gate passes.



Loops until it passes — gated by you

Hands the recommended fix to an AI engineer that edits your agent, then re-runs evaluate. It loops — fix → re-build → re-evaluate — until the agent passes. **Nothing changes without your go-ahead.**

💬 YOU CAN SAY

“apply the recommended fix to the support-triage agent”

“make the support-triage agent pass its evals”

📦 PRODUCES **the updated agent + a fresh scorecard**